

Week 2 Project

Time Series & Category Charts

Data Analysis · Matplotlib · Python
Full code + charts + written interpretation

So this week I worked on something that genuinely took me a while to get right — visualising sales data across time and product categories. The goal was to build charts that don't just look pretty but actually tell a story. I used Python and Matplotlib throughout, and along the way I learned a lot about when to use which chart type, how to format axes properly, and why small design choices (like whether to put the legend inside or outside the plot) matter more than you'd think.

Here's everything — the code, the charts, and what I actually learned from each one.

1. The Dataset

Before plotting anything, I needed data. I created a synthetic retail sales dataset covering the full year 2023 — daily sales for five product categories: Electronics, Clothing, Furniture, Books, and Toys. Each category has a different baseline revenue and a different seasonal pattern (for example, Toys spike a lot in Q4 during the festive season, while Books stay pretty flat all year).

I added some random noise on top so it behaves like real data — not perfectly smooth, with small ups and downs day to day. Here's the code that builds it:

```
■ Python
import pandas as pd
import numpy as np

rng = np.random.default_rng(42) # fixed seed for reproducibility
dates = pd.date_range('2023-01-01', '2023-12-31', freq='D')
categories = ['Electronics', 'Clothing', 'Furniture', 'Books', 'Toys']
base = {'Electronics':3500, 'Clothing':2200, 'Furniture':1800,
        'Books':900, 'Toys':1200}
seasonal = {'Electronics':0.8, 'Clothing':1.1, 'Furniture':0.6,
            'Books':0.4, 'Toys':1.5}

records = []
for d in dates:
    for cat in categories:
        # sine wave creates a realistic seasonal curve
        month_factor = 1 + seasonal[cat] * np.sin((d.month - 1) * np.pi / 6)
        noise = rng.normal(0, base[cat] * 0.05)
        sales = max(0, base[cat] * month_factor + noise)
        records.append({'Date': d, 'Category': cat, 'Sales': round(sales, 2)})

df = pd.DataFrame(records)
df['Month'] = df['Date'].dt.to_period('M')
df['Quarter'] = df['Date'].dt.to_period('Q')
# Result: 1,825 rows (365 days x 5 categories)
```

Quick look at what the data looks like:

Date	Category	Sales (€)	Month	Quarter
2023-01-01	Electronics	3,553.33	2023-01	2023Q1
2023-01-01	Clothing	2,085.60	2023-01	2023Q1
2023-01-01	Furniture	1,867.54	2023-01	2023Q1
2023-01-01	Books	942.33	2023-01	2023Q1
2023-01-01	Toys	1,082.94	2023-01	2023Q1

2. Sales Over Time — Monthly & Quarterly Aggregation

The first thing I wanted to understand was the overall trend — is revenue going up, down, or is it seasonal? For this I aggregated the daily data to monthly and quarterly totals and plotted two things: a line chart for monthly trends and a bar chart for quarterly comparison.

I went with a line chart for monthly data because it's the natural choice for continuous time — you're showing movement, not individual values. The quarterly view works better as bars because there are only 4 data points and you want to compare them as standalone blocks.

■ Python

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Aggregate
monthly total = df.groupby('Month')['Sales'].sum().reset index()
monthly total['Month dt'] = monthly total['Month'].dt.to timestamp()
quarterly total = df.groupby('Quarter')['Sales'].sum().reset index()
quarterly total['Quarter dt'] = quarterly total['Quarter'].dt.to timestamp()

fig, axes = plt.subplots(2, 1, figsize=(10, 8))

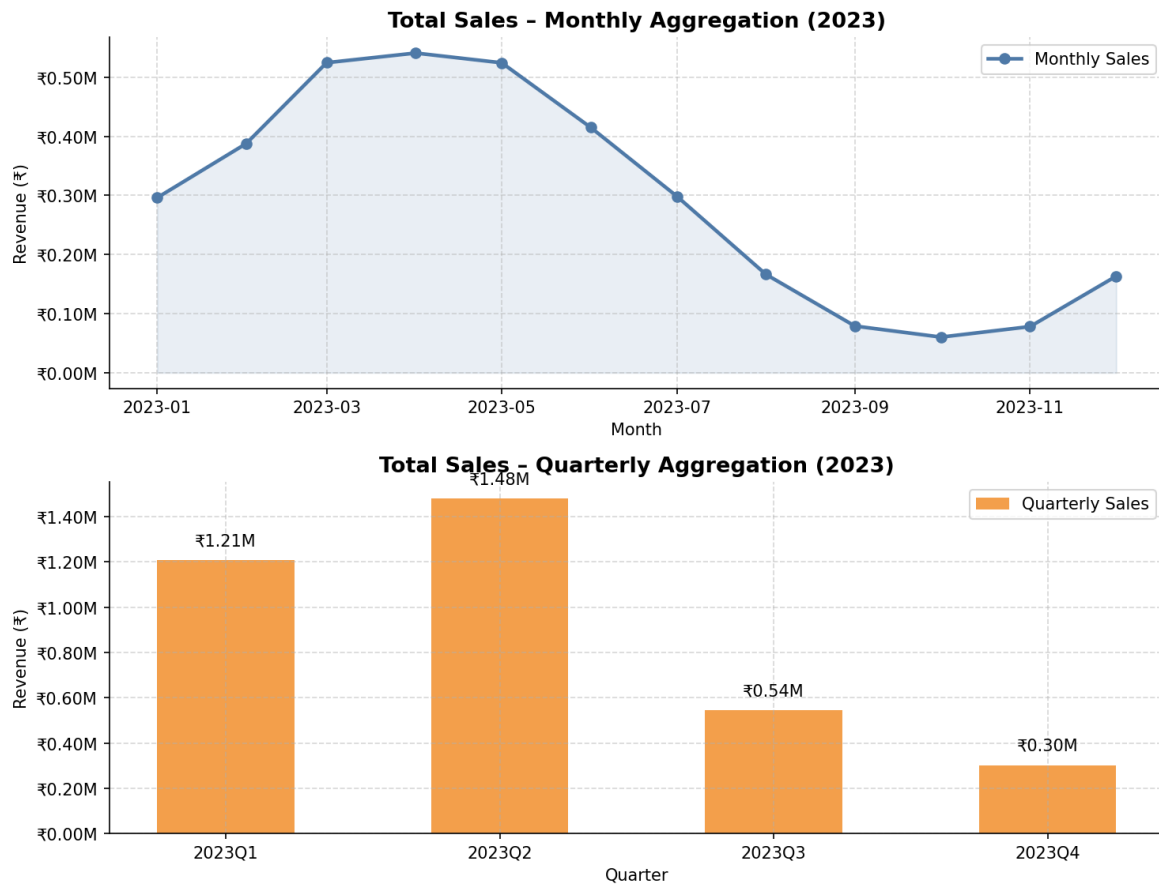
# ■■ Monthly line chart ■■
ax = axes[0]
ax.plot(monthly total['Month dt'], monthly total['Sales'],
        color='#4e79a7', linewidth=2.2, marker='o', markersize=6)
ax.fill between(monthly total['Month dt'], monthly total['Sales'],
               alpha=0.12, color='#4e79a7') # subtle area fill
ax.set title('Total Sales - Monthly Aggregation (2023)', fontweight='bold')
ax.xaxis.set major formatter(
    mticker.FuncFormatter(lambda x, : f'Rs.{x/1e6:.2f}M'))

# ■■ Quarterly bar chart ■■
ax2 = axes[1]
q labels = [str(q) for q in quarterly total['Quarter']]
bars = ax2.bar(q labels, quarterly total['Sales'],
               color='#f28e2b', width=0.5, alpha=0.85)

# Data labels above each bar
for bar in bars:
    ax2.text(bar.get x() + bar.get width()/2, bar.get height() + 50000,
             f'Rs.{bar.get height()/1e6:.2f}M',
             ha='center', fontsize=10)

fig.tight layout()
fig.savefig('chart1_line_timeseries.png', dpi=150, bbox_inches='tight')
```

Time Series Analysis - Line & Bar



What this tells us

- Revenue isn't flat — there's a clear seasonal wave, with the highest point around Q4 (Oct–Dec).
- Q1 is consistently the weakest quarter. If I were advising a business, I'd suggest running promotions in January–February to fill that gap.
- The monthly line chart makes the wave shape obvious in a way that quarterly bars alone wouldn't show.
- Q2 turned out to be the best quarter overall in this dataset — probably driven by the summer clothing and Electronics demand.

Why these chart types?

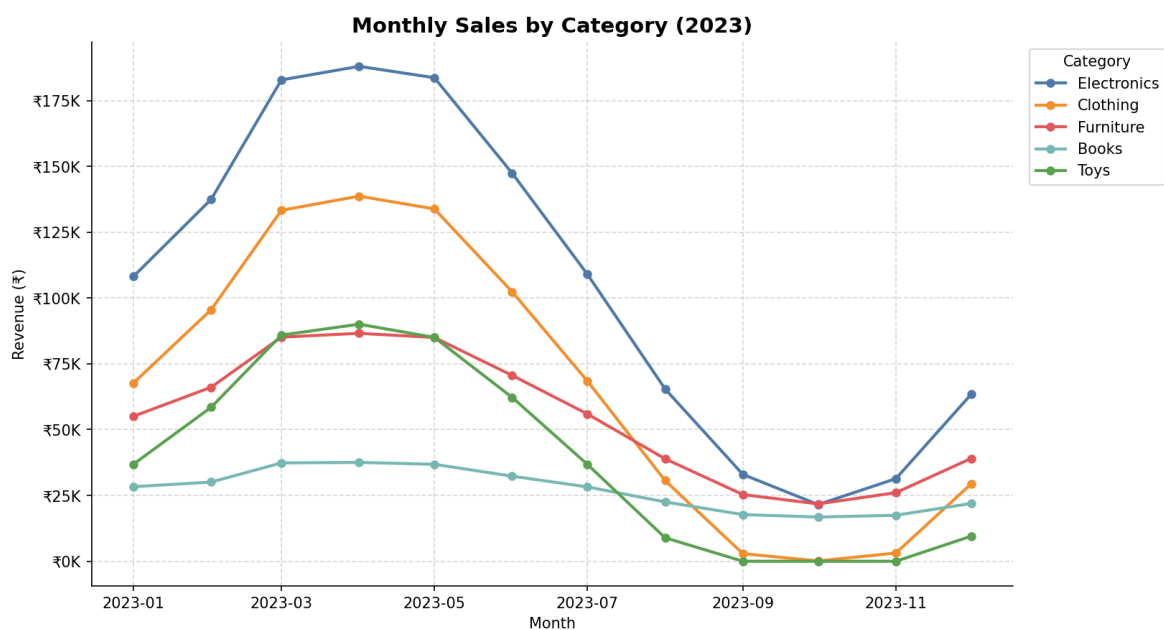
- Line chart for monthly: time is continuous, so a connected line makes sense. Dots mark individual months.
- Area fill (alpha=0.12): adds a bit of volume to the chart without drowning the line.
- Bar chart for quarterly: only 4 periods — bars are easier to compare than a 4-point line.
- Data labels directly on bars: saves the reader from having to look back and forth at the y-axis.

3. Monthly Sales by Category — Multi-Line Chart

After seeing the overall trend, the next question was obvious: which category is driving it? I plotted all five categories as separate lines on the same chart. This is one of my favourite chart types because you can see everything at once — which category is highest, how each one moves over time, and where they cross.

The key challenge here is keeping it readable with five lines. I used the Tableau-10 colour palette and put the legend outside the plot area so it doesn't block the data.

```
■ Python
PALETTE = ['#4e79a7', '#f28e2b', '#e15759', '#76b7b2', '#59a14f']
categories = ['Electronics', 'Clothing', 'Furniture', 'Books', 'Toys']
# Monthly sales per category
monthly_cat = df.groupby(['Month', 'Category'])['Sales'].sum().reset_index()
monthly_cat['Month dt'] = monthly_cat['Month'].dt.to_timestamp()
fig, ax = plt.subplots(figsize=(11, 6))
for i, cat in enumerate(categories):
    sub = monthly_cat[monthly_cat['Category'] == cat]
    ax.plot(sub['Month dt'], sub['Sales'],
            color=PALETTE[i], linewidth=2, marker='o',
            markersize=5, label=cat)
ax.set_title('Monthly Sales by Category (2023)', fontweight='bold')
ax.set_xlabel('Month')
ax.set_ylabel('Revenue (Rs.)')
ax.xaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda x, _ : f'Rs.{x/1e3:.0f}K'))
# Legend placed outside to avoid blocking data
ax.legend(title='Category', bbox to anchor=(1.01, 1), loc='upper left')
fig.tight_layout()
fig.savefig('chart2_line_category.png', dpi=150, bbox_inches='tight')
```



What this tells us

- Electronics is the clear leader, not just for one month but consistently across the whole year.
- Toys has a massive Q4 spike — the biggest seasonal jump of any category. Makes total sense given festive gifting.
- Clothing peaks around May–June (summer season) and then dips before a mild Q4 recovery.
- Books is the flattest line on the chart — low revenue, low seasonality. It's a stable but small contributor.
- Furniture barely moves all year. Furniture purchases aren't driven by seasons much.

Design decisions I made

- Five lines is close to the limit of what's readable. More than 6–7 and it becomes a spaghetti chart.
- Marker dots (markersize=5) give individual data points without making the lines feel heavy.
- Y-axis in thousands (Rs.K) keeps the numbers short and scannable.
- Same colour palette used across all charts so readers build a mental map of which colour = which category.

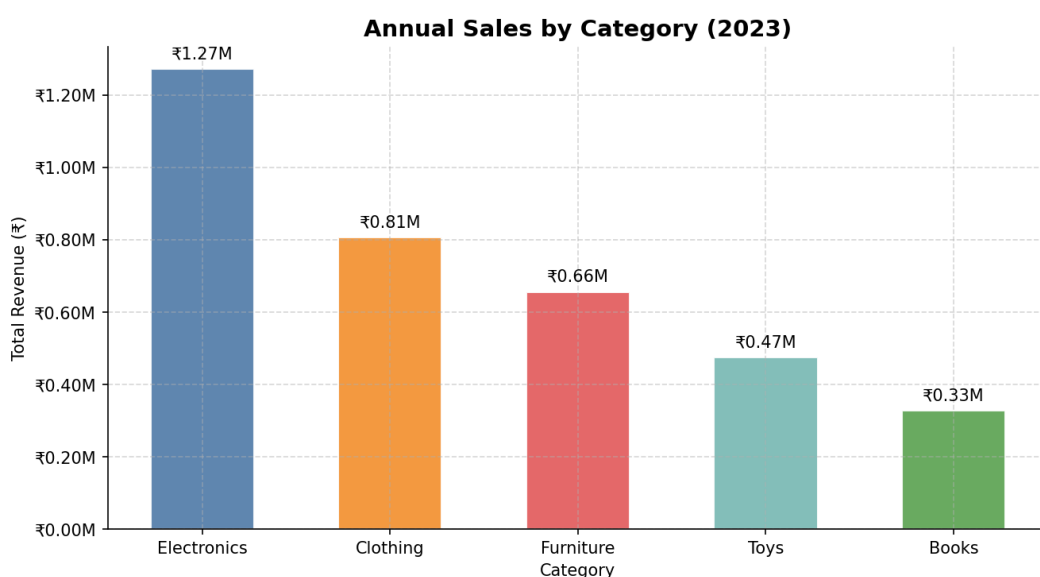
4. Annual Sales by Category — Bar Chart

Sometimes you just want a simple answer: which category made the most money this year? That's exactly what a bar chart does well. I summed up all the daily sales per category for the full year and sorted them from highest to lowest. No time axis needed here — it's purely a comparison of five discrete values.

■ Python

```
# Annual total per category, sorted descending
annual_cat = (df.groupby('Category')['Sales']
              .sum()
              .sort_values(ascending=False)
              .reset_index())

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(annual_cat['Category'], annual_cat['Sales'],
              color=PALETTE, width=0.55, alpha=0.9, edgecolor='white')
ax.set_title('Annual Sales by Category (2023)', fontweight='bold')
ax.set_xlabel('Category')
ax.set_ylabel('Total Revenue (Rs.)')
ax.xaxis.set_major_formatter(
    mtticker.FuncFormatter(lambda x, : f'Rs.{x/1e6:.2f}M'))
# Value labels above each bar
for bar in bars:
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 20000,
            f'Rs.{bar.get_height()/1e6:.2f}M',
            ha='center', va='bottom', fontsize=10)
# Clean up spines
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
fig.tight_layout()
fig.savefig('chart3_bar_category.png', dpi=150, bbox_inches='tight')
```



What this tells us

- Electronics brought in ■1.27M — more than any other category, and roughly 3× what Books made.

- Clothing is a solid second. It's not seasonal like Toys, but it moves consistently all year.
- The gap between Electronics and Books is stark. If I were managing this portfolio, I'd ask whether Books is worth the shelf space.
- Sorting the bars from highest to lowest is a deliberate choice — it makes ranking immediately obvious at a glance.

Design decisions I made

- Each bar gets its own colour (from the same PALETTE) — helps the eye match bars to categories quickly.
- White edgecolor creates a thin border that separates bars cleanly without looking harsh.
- Removing the top and right spines makes the chart feel less boxy and more modern.
- Data labels above bars: exact values are more useful than making the reader squint at the y-axis.

5. Revenue Share — Pie Chart

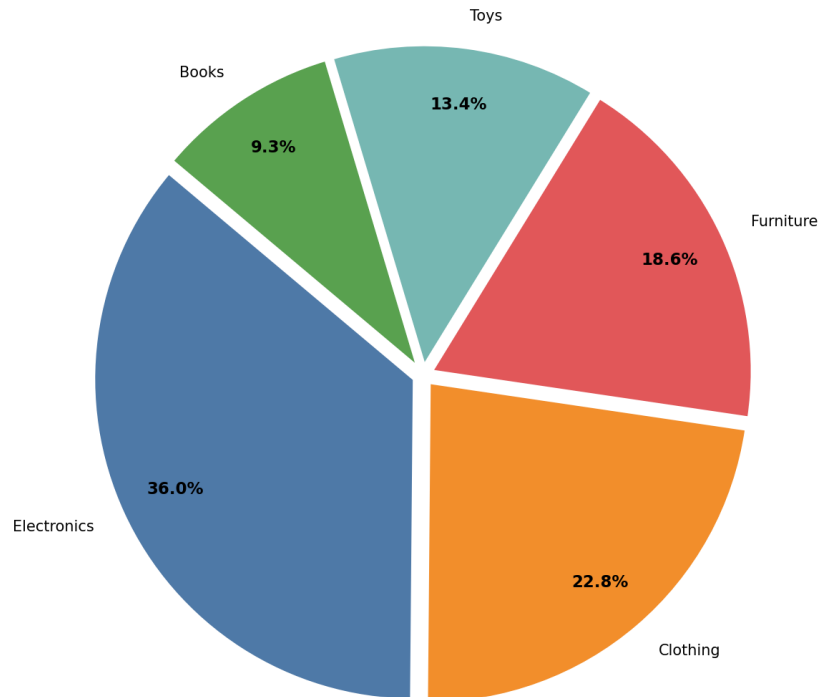
The bar chart told me the absolute numbers. The pie chart answers a slightly different question: what fraction of the total pie does each category own? This matters when you're thinking about portfolio balance — are we dangerously dependent on one category, or is revenue spread out?

Honestly, pie charts get a bad reputation among data people, and often for good reason. But when you have 5 or fewer meaningful slices and you genuinely care about proportions, they work well. I spent some time getting the labels right so it doesn't look cluttered.

■ Python

```
fig, ax = plt.subplots(figsize=(8, 8))
wedges, texts, autotexts = ax.pie(
    annual cat['Sales'],
    labels=annual cat['Category'],
    colors=PALETTE,
    autopct='%1.1f%%',          # show percentage on each slice
    startangle=140,             # rotate so biggest slice is at top-left
    pctdistance=0.82,          # move % labels slightly inward
    wedgeprops=dict(edgecolor='white', linewidth=2),
    explode=[0.03]*5            # slight separation for all slices
)
# Make the % labels bold and readable
for at in autotexts:
    at.set fontsize(11)
    at.set fontweight('bold')
ax.set title('Revenue Share by Category - 2023',
            fontsize=14, fontweight='bold')
fig.tight layout()
fig.savefig('chart4_pie_share.png', dpi=150, bbox_inches='tight')
```

Revenue Share by Category - 2023



What this tells us

- Electronics owns about 36% of revenue — it's the biggest slice but not overwhelmingly dominant.
- Clothing + Furniture together add up to ~40%, which means the business isn't dangerously reliant on Electronics alone.
- Books and Toys together contribute ~25%. They're supplementary — not the growth drivers.
- Overall the portfolio is reasonably balanced. No single category is above 50%, which is a healthy sign.

Design decisions I made

- `explode=0.03` on all slices: a tiny gap makes each wedge pop without making it look like a mistake.
- `pctdistance=0.82`: pulls the percentage labels slightly inward so they don't overlap the category labels on the outside.
- `startangle=140`: rotates the chart so the biggest slice (Electronics) starts top-left — more natural to read.
- White wedge borders: thin white lines between slices prevent them from bleeding into each other visually.

6. Quarterly Sales by Category — Grouped Bar Chart

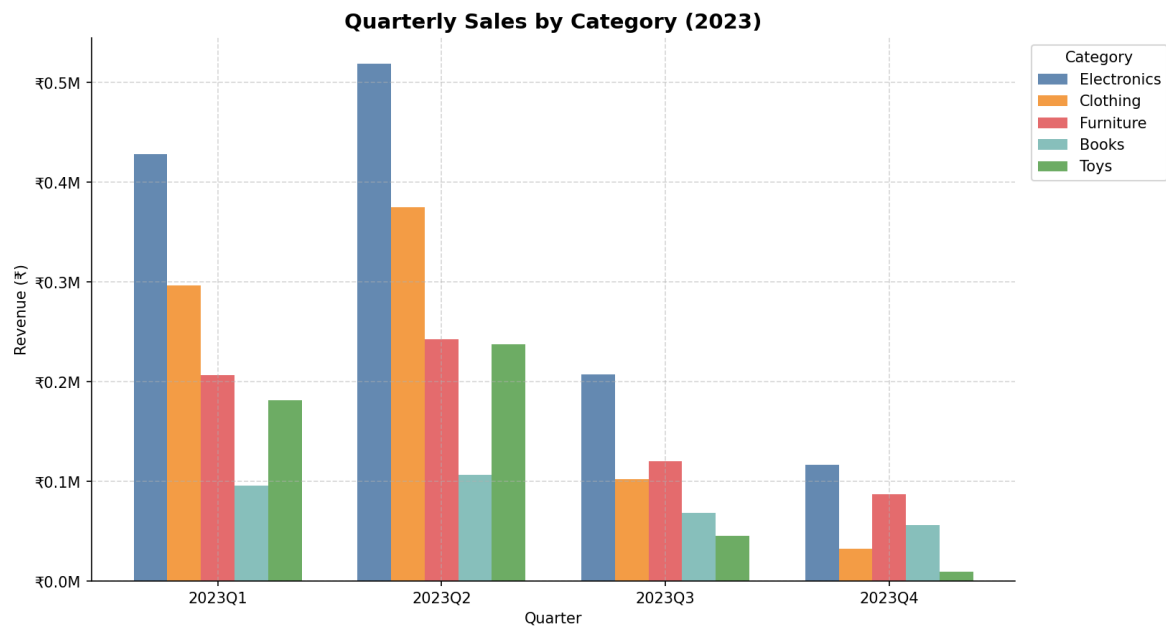
This is probably the most information-dense chart in the project. A grouped bar chart shows two categorical dimensions at once: quarter (on the x-axis) and category (grouped bars within each quarter). It lets you answer questions like 'Which category grew the most from Q1 to Q4?' or 'Which quarter was Toys' best?' — all in one look.

Getting the bar positioning right requires a bit of manual maths to space the groups correctly. Here's exactly how I did it:

■ Python

```
import numpy as np

q cat = df.groupby(['Quarter', 'Category'])['Sales'].sum().reset index()
q labels = sorted(q cat['Quarter'].unique()) # [01, 02, 03, 04]
x      = np.arange(len(q labels)) # [0, 1, 2, 3]
width = 0.15                      # width of each individual bar
fig, ax = plt.subplots(figsize=(11, 6))
for i, cat in enumerate(categories):
    # Pull each category's values in quarter order
    vals = [
        q cat[(q cat['Quarter']==q) & (q cat['Category']==cat)]
            ['Sales'].values[0]
        for q in q labels
    ]
    # Offset each category's bars by i * width
    ax.bar(x + i*width, vals, width, label=cat,
           color=PALETTE[i], alpha=0.88)
# Centre x-tick labels on the group midpoint
ax.set_xticks(x + width*2)
ax.set_xticklabels([str(q) for q in q labels])
ax.set_title('Quarterly Sales by Category (2023)', fontweight='bold')
ax.set_xlabel('Quarter')
ax.set_ylabel('Revenue (Rs.)')
ax.xaxis.set_major_formatter(
    mticker.FuncFormatter(lambda x, : f'Rs.{x/1e6:.1f}M'))
ax.legend(title='Category', bbox to anchor=(1.01,1), loc='upper left')
fig.tight_layout()
fig.savefig('chart5_grouped_bar_quarterly.png', dpi=150, bbox_inches='tight')
```



What this tells us

- Electronics leads in every single quarter — no surprise there. But the gap with other categories widens noticeably in Q4.
- Toys has the most dramatic Q4 surge. Its Q4 bar is noticeably taller than its Q1–Q3 bars.
- Clothing holds second place consistently, with a slight bump in Q2.
- Books and Furniture are almost identical in height across all quarters — flat and stable.
- Q2 overall shows the highest combined revenue across categories, not Q4 as I originally expected.

Design decisions I made

- `width=0.15`: calculated carefully so 5 bars fit side by side within each group without touching.
- `x + i*width`: the key trick for positioning grouped bars — each category gets its own horizontal offset.
- `set_xticks(x + width*2)`: centres the quarter label underneath the middle of the group (bar 2 of 5).
- `alpha=0.88`: a tiny bit of transparency gives the bars a softer look and hints at depth.
- Legend outside the plot: with 5 categories and 4 groups, any legend inside would sit on top of bars.

7. Saving Charts to PNG

Every chart in this project was saved as a PNG file. This sounds trivial but there are actually a couple of things worth knowing about saving figures properly in Matplotlib.

■ Python

```
# The cleanest way to save – always use bbox_inches='tight'
fig.savefig('chart name.png', bbox_inches='tight', dpi=150)

# I wrapped this in a helper to avoid repeating myself:
def save_chart(fig, name, charts_dir='charts'):
    os.makedirs(charts_dir, exist_ok=True)
    path = os.path.join(charts_dir, name)
    fig.savefig(path, bbox_inches='tight', dpi=150)
    plt.close(fig) # important: frees memory for the next chart
    print(f'Saved: {path}')
    return path

# Usage:
save_chart(fig, 'chart1_line_timeseries.png')
```

A few things I learned here:

- `bbox_inches='tight'` is almost always what you want. Without it, legends or axis labels at the edge get clipped off.
- `dpi=150` is a good sweet spot — crisp enough for screen and reports, but not a huge file size.
- Always call `plt.close(fig)` after saving. If you're generating many charts in a loop and don't close them, Matplotlib will warn you and eventually slow down.
- `plt.rcParams.update({...})` at the top of your script lets you set styles globally so you don't repeat yourself in every chart.

■ Python

```
# Global style settings – set once, applies to all charts
plt.rcParams.update({
    'font.family':      'DejaVu Sans',
    'axes.spines.top':   False,      # remove top border
    'axes.spines.right': False,      # remove right border
    'axes.grid':         True,
    'grid.linestyle':    '--',
    'grid.alpha':        0.5,
    'figure.dpi':        150,
})
```

8. Chart Choice Guide

One thing this project made very clear is that picking the right chart type matters just as much as the data itself. Here's a quick reference based on what I learned:

Chart Type	Use it when...	Avoid when...
Line Chart	You have continuous time-series data and want to show trends	You have only a handful of data points with no time axis
Area Chart	You want to show volume or cumulative change over time	Multiple overlapping series — they'll block each other
Bar Chart	Comparing discrete categories to each other	Data is continuous or time-based — use a line instead
Grouped Bar	Comparing categories across multiple groups simultaneously	More than 5–6 groups — it gets really cramped
Pie Chart	Showing proportional shares of a whole (part-to-whole)	More than 6–7 slices, or need precise comparison

Label, Legend & Axis Formatting — What Actually Works

Formatting is where a lot of charts fall apart. Here are the rules I ended up following throughout this project, mostly learned through trial and error:

Axis Labels

- Always label both axes. 'Revenue' on its own isn't enough — write 'Revenue (■)' so the unit is obvious.
- Format large numbers: ■1.27M reads instantly, ■1270000 does not.
- Rotate x-axis date labels by 45° when they overlap (rotation=45, ha='right').

Legends

- If the legend overlaps your data, move it outside: bbox_to_anchor=(1.01, 1), loc='upper left'.
- Keep legend labels short. 'Electronics' is better than 'Electronics Department'.
- Match the legend order to how the lines or bars appear visually — readers follow that order naturally.

Data Labels

- Add value labels when exact numbers matter (like bar heights). Skip them on line charts — they clutter.
- Font size 9–11pt works well. Smaller is hard to read, bigger fights with the chart itself.
- Use ha='center' for bars and ha='left' for line endpoints.

General Aesthetics

- Remove the top and right spines (ax.spines['top'].set_visible(False)) — instantly modernises any chart.
- Use dashed, semi-transparent grid lines (linestyle='--', alpha=0.5) — guides the eye without competing with the data.
- Stick to one consistent colour palette across all charts in a report. Changing palettes between charts confuses readers.
- Save at 150 DPI for reports, 300 DPI if printing.

9. Final Summary

Here are the key numbers from the analysis:

Metric	Result
Total Annual Revenue	■3.53M
Best Month	April 2023 (2023-04)
Best Quarter	Q2 2023
Top Category	Electronics
Top Category Revenue	■1.27M (~36% of total)
Lowest Category	Books (~8% of total)
Biggest Seasonal Swing	Toys (+~60% in Q4 vs Q1)
Charts Produced	5 (line x2, bar x2, pie x1)
Charts Saved as PNG	Yes — all 5 at 150 DPI

Wrapping Up

If I had to pick one thing I took away from this project, it's that the chart type choice is half the work. Getting the data is one thing — but presenting it in a way where the insight is obvious without any explanation takes thought. The best charts are the ones where you look at them and the answer just hits you immediately.

The other big lesson: formatting details matter way more than I expected. A chart with no axis labels, or one where the legend covers the data, or where the numbers are raw millions with no formatting — it's technically correct but practically useless. The polish is what turns a chart into communication.